*** Republic of Tunisia ***

Minister of Higher Education, Scientific Research,
and Information and Communication Technology

General Directorate of Technological Studies

Higher Institute of Technological Studies of Beja
Mechanical Engineering Department

Graduation Project Dissertation

to obtain the National Diploma of Applied License in Mechanical Engineering

Dev and Deployment of an automated real-time monitoring system for TESCA group

Carried out by :

Fedi Amdouni & Firas Ouesleti

Defended on 11/06/2026, in front of the committee members :

- **Mrs. Maroua Ben Brahim** — President
- **Mrs. Sihem Dridi** — Reviewer
- **Mr . Radhouane Aloui** — University supervisor
- **Mr . Fethi Yahyaoui** — Company supervisor

End of studies internship completed at

Host company TS2I Technique Solution et Innovation Industriel

Client company tesca

Academic year 2025-2026

Contents

List of Figures

List of Tables

General Introduction

In today's competitive industrial landscape, production efficiency is no longer a luxury, it is a necessity. For manufacturing firms, production efficiency is essential because their production process faces numerous challenges, and they must be ready at all times to optimize production output, reduce wastage and mitigate any disturbances that may arise within the production floor. One of the critical indicators of the effectiveness of production processes is the Overall Equipment Effectiveness (OEE), known in French as the *Taux de Rendement Synthétique* (TRS), stands out as one of the most comprehensive metrics, capturing the combined impact of availability, performance and quality losses in the production process. [1].

The automotive industry, governed by rigorous quality standards such as IATF 16949 [2], is particularly demanding in this regard. Tunisia has established itself over the decades as a competitive industrial hub for automotive suppliers, attracting major international players thanks to its geographic proximity to Europe, its cost-efficient production and skilled workforce [3]. Among these companies is TESCA Group, an international manufacturer specializing in vehicle interior components, with several production sites operating across Tunisia [4].

TESCA Group has always prioritized continuous improvement. Better visibility into production matters—detecting stoppages quickly, understanding what causes them and responding fast directly impact competitiveness. To achieve this, they developed a production monitoring application that automates data collection and OEE calculation.

But operational use revealed real problems with the Node-RED setup. The dashboard can't be customized, visualizations are limited and it doesn't scale. Integration is restricted. The company also has no mechanical documentation, which means maintenance teams can't diagnose equipment problems or plan maintenance work properly. These gaps stop the system from supporting TESCA Group's improvement goals.

Our Final Year Project tackled these issues in three ways. We built a modern dashboard to replace the old interface—one that's easier to use and lets teams customize what they see. We rewrote the backend from scratch on a proper server, which means better performance and reliability. We also put together complete documentation for the mechanical systems so maintenance crews can actually fix things without guessing. The result is a monitoring system that gives TESCA Group real visibility into what's happening on the floor, helps them spot problems faster and makes their continuous improvement work actually possible.

This report summarizes all the work completed during this Final Year Project.It documents five key areas of development: first, preliminary analysis presenting the two hosting organizations and covering the project context and the limitations. Second,the backend architecture migration and automation programming, and HMI logic development. Third, the development of a modern, dedicated frontend dashboard with professional data visualization and customizable layouts. Fourth, the database optimization to improve system performance, reliability and scalability. And finally, the creation of comprehensive technical documentation for mechanical systems to support long-term operations.

Chapter 1

Preliminary analysis

Introduction

The first chapter of this report aims to provide an overview of the host and the client companies, present the case study by highlighting the problem to be addressed and introduce the modeling language used for the project. This chapter sets the stage for the subsequent sections by establishing the context, significance and scope of the actions undertaken in the next chapters.

1 Presentation of the enterprises

1.1 Technique Solution Et Innovation Industriel (TS2I)

TS2I is a leading company in the field of specialised machines fabrication, automation, industrial maintenance, and Tampographique service. With a strong commitment to quality, innovation and customer satisfaction, TS2I has established itself as a trusted partner for various industries. The following provides an overview of TS2I and highlights its expertise in fabrication machines, automation, industrial maintenance and pad printing services.

- **Fabrication of specialised machines:** TS2I specializes in the design, development and production of specialised machines. These machines are tailored to meet the unique requirements of clients in diverse industries such as automotive, aerospace, electronics, and more. With a team of experienced engineers and technicians, TS2I leverages cutting-edge technologies to deliver custom fabrication machines that optimize productivity, efficiency and quality. Whether it's automated assembly lines, robotic systems or specialized machining equipment, TS2I ensures that each machine is built to the highest standards.

- **Automation:** Automation plays a crucial role in enhancing productivity and reducing operational costs. TS2I offers comprehensive automation solutions to help businesses automate their processes. From simple control systems to complex integrated automation solutions, TS2I designs and implements tailored solutions based on clients requirements. Their expertise in PLC programming, HMI design, robotics and motion control allows them to deliver efficient and reliable automation solutions that streamline operations and improve overall efficiency.

- **Industrial maintenance:** TS2I understands the importance of maintaining industrial equipment to ensure optimal performance and minimize downtime. The company offers comprehensive maintenance services to address the unique needs of its clients. Whether it's preventive maintenance, troubleshooting or equipment repair, TS2I's skilled technicians and engineers are well-equipped to handle a wide range of industrial machinery. By providing regular maintenance, TS2I helps clients avoid costly breakdowns, increase equipment lifespan and maximize productivity.

- **Tampographique service:** In addition to its fabrication machines and automation expertise, TS2I also offers a specialized service in tampography or pad printing. Tampography is a versatile printing technique used for applying designs or logos onto various surfaces including plastics, metals, ceramics and glass. TS2I's pad printing service ensures high-quality and precise printing, providing clients with branding solutions that are durable and visually appealing. Whether it's product customization, promotional items or industrial labeling, TS2I's pad printing service offers reliable and efficient solutions.

1.2 TESCA

1.2.1 Presentation of the company

Tesca Group is a family-run company with a background in textiles that has successfully ventured into the automotive industry, specifically automotive seating. Emulating the approach of a French automotive manufacturing house, Tesca Group meticulously crafts each piece, from its atelier to its global production sites[4]. The company strives to embody the same creative audacity and prides itself on being a partner to major automotive manufacturers like:

Figure 1: Tesca's partners

With a clientele consisting of demanding visionaries who shape the future of mobility, Tesca Group positions itself as a reliable and flexible partner, accompanying its clients in their pursuit of innovation and competitiveness. The company believes that adopting a long-term and eco-friendly vision is crucial for sustainable growth, constantly pushing for excellence rather than settling for mediocrity[4].

- **Sustainability:** A concrete and integral part of Tesca Group's approach. The company is committed to sustainable development and actively incorporates environmental and social values into its products, services, and activities within the automotive industry. Since 2004, Tesca Group has been dedicated to controlling its ecological footprint, and the executive management maintains and expands upon these efforts. The company adheres to the values and principles of the Global Compact through its Ethics Charter, to obtain ISO 14001 certification for all its sites[4].

- **Environmental policy:** Tesca Group's environmental policy revolves around five major directions: behaving in an environmentally responsible manner, controlling the industrial waste, optimizing energy consumption and natural resources, participating in the reduction of the ecological footprint throughout the design and production processes and ensuring compliance with environmental regulations and requirements[4].

- **Innovation:** A driving force within Tesca Group, with a constant emphasis on generating new ideas and executing them effectively. The company's research programs focus on key themes in the automotive industry, such as weight reduction, optimized part design and ecological footprint expertise. While prioritizing user comfort, functionality and safety. Tesca Group holds numerous patents in the conception, design and manufacturing of automotive parts and seat components, including headrests, armrests, upholstery, padding and "smart textiles"[4].

- **Industrial excellence:** A cornerstone of Tesca Group's strategy, whether through its local production sites or research centers. The company strives to provide reliable processes that meet the evolving market expectations worldwide. Tesca Group empowers its teams through a culture of continuous improvement, with competitiveness, efficiency and exceptional service forming the foundation of its industrial success. SPRINT (Tesca's Industrial Production System) is employed to engage collaborators at all levels, aiming to standardize procedures and ensure perpetual progress in industrial performance. Shared values among Tesca Group employees include excellence, self-discipline, commitment, autonomy and pragmatism[4].

- **Quality:** Tesca Group's Quality policy aligns with five major directions: meeting commitments to conformity, cost and deadlines; aligning industrial and purchasing performance with the highest quality criteria; standardizing existing processes to offer innovative products; prioritizing process control and prevention; and continuously enhancing the skills of its teams[4].

- **History:** In terms of its history, Tesca Group has a notable timeline of achievements. Starting in 1960 with the provision of Citroën 2CV roofs, the company went on to pioneer and apply the Foam In Place technology in 1978. In 1983, Tesca Group established its Design Studio in Paris, followed by the creation of the CERA research and development center in Reims in 1993. By the year 2000, the company had expanded its production capabilities worldwide. In 2016, Tesca Group underwent a significant transformation, becoming Trèves TSC[4].

Overall, Tesca Group has established itself as a reputable partner to major automotive manufacturers driven by a commitment to sustainability, innovation, industrial excellence and quality. Through its diverse range of products and services, the company aims to shape the future[4].

1.2.2 Technical file

The following table presents the technical file of TESCA.

Table 1: Technical file of TESCA

Company name	**TESCA**
Sector	Vehicle interior equipment
Creation	2019
Legal form	Foreign company
Staff	179 employees
Address	Grombalia Industrial Zone – Nabeul– Tunisia

1.2.3 Organizational chart

The organizational chart shown in Figure 2 presents the distribution of positions and functions within the Tesca enterprise.

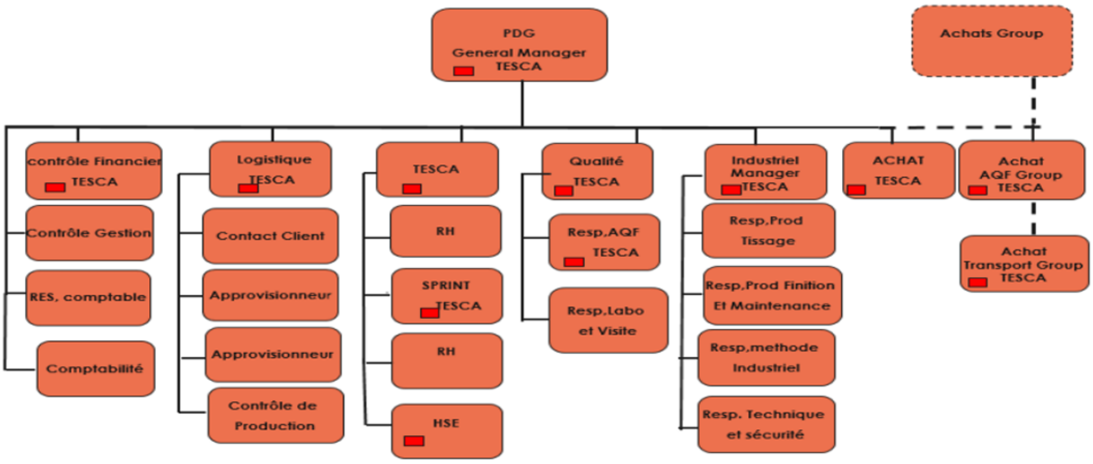

Figure 2: Organizational chart of TESCA

2 Case study

2.1 Overview of the Existing System

The existing production monitoring system was developed to replace manual paper-based stop recording with automated digital tracking. The system consists of three components working together: a Delta DVP PLC installed on the fabric inspection machine, a Node-RED platform serving as the intermediary that reads PLC data and communicates with the database, and a MySQL database storing all production information.

Before this system existed, operators manually recorded all machine stops, causes, and durations on paper during their shifts. This process was prone to errors, made it hard to monitor, and could not conduct real-time analysis. In the new system, the PLC senses when the machine is stopped; the operator selects the reason for the stoppage via a tablet, and the system automatically computes the time the machine takes to start.

There are two main users of the system. Operators on the production floor use a tablet application that displays real-time machine speed, yardage measurements, lighting controls, and a dropdown menu to select stop causes. This interface updates continuously and provides immediate feedback, making it practical for shop-floor use. Managers access a web interface where they can filter data by date and shift team, view a table of all stops with their causes and durations, see a bar chart showing which causes are most frequent, and view the daily OEE (Overall Equipment Effectiveness) metric that measures productive time versus downtime.

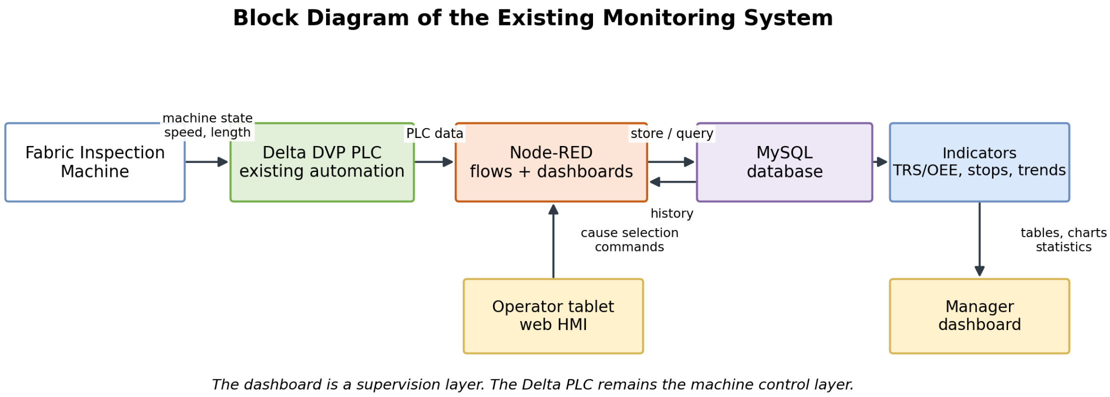

Figure 3: Block Diagram of the Production Monitoring System

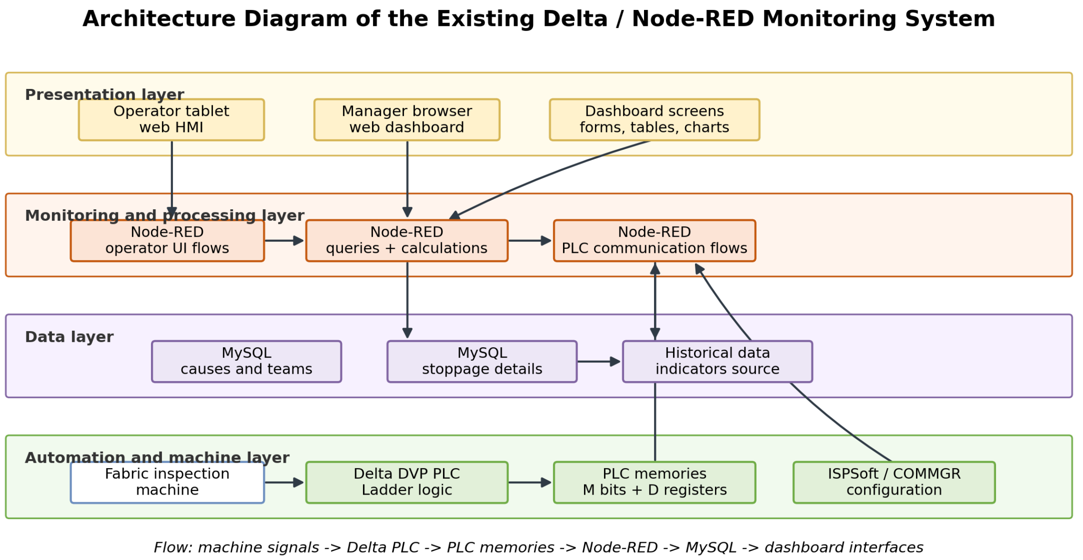

Figure 4: Architecture Diagram of the Production Monitoring System

2.2 Criticism of the existing system

2.2.1 Limited Data Analysis

The existing system successfully records stops but provides limited analytical capability. The manager interface shows a bar chart of stop causes and a table of stops for the selected period, but it does not show trends over time. A manager viewing one week of data sees aggregated numbers but cannot easily see whether the situation is improving or deteriorating day by day. The interface does not enable comparison between teams or shifts, making it difficult to identify performance patterns.

2.2.2 No Real-Time Manager Updates

A significant limitation is that the manager web interface does not automatically update. When a manager opens the dashboard to check production status, they must manually click a filter button to retrieve current data. If new stops occur after they view the data, these are not automatically displayed—the manager sees the situation as it was at their last manual refresh. In production environments, delays in visibility can mean slower response to problems.

2.2.3 Limited Performance Metrics

The system primarily reports OEE and stop causes. Modern production monitoring typically tracks additional metrics to provide a complete picture. The system does not show trends in OEE over time, separate availability from performance metrics, or provide any predictive capability to identify potential problems before they occur. A manager cannot easily identify whether equipment health is improving or deteriorating, or whether specific causes show patterns that might indicate underlying issues.

2.2.4 Inflexible User Interface and Limited Access

All managers see the same view regardless of their role or needs. The interface is designed for desktop computers and is not optimized for mobile devices, meaning managers cannot check production status from a smartphone while away from their office. There is also no concept of custom dashboards where different users might want different information.

2.2.5 Data Quality and Verification

The system relies on operators to manually select stop causes from a dropdown menu. There is potential for human error or inconsistency in cause selection. Once a cause is recorded, there is no verification process or audit trail to show who entered the data or when. If a data discrepancy is discovered later, there is no way to trace back what happened.

2.2.6 Lack of Security and Integration

The existing system has no user authentication or access control. Anyone with the IP address can access all production data without logging in. The system also cannot export data to other formats or systems. If a manager needs to perform advanced analysis in Excel or if another department needs production data, it must be manually copied or requested.

2.2.7 Scalability Concerns

The Node-RED platform handles all functions in a monolithic design: reading from the PLC, storing in the database, and serving both the operator tablet and manager web interface. As data volume grows or additional machines are added, this architecture may face performance or reliability issues, since any slowdown in one function could impact others.

2.3 Specifications Book

The system to be developed shall improve the existing monitoring dashboard by strengthening real-time supervision, data reliability, traceability, analysis capabilities and user access control. It shall continue to use the production data made available by the existing industrial environment and shall not disturb the machine control logic executed by the Delta PLC.

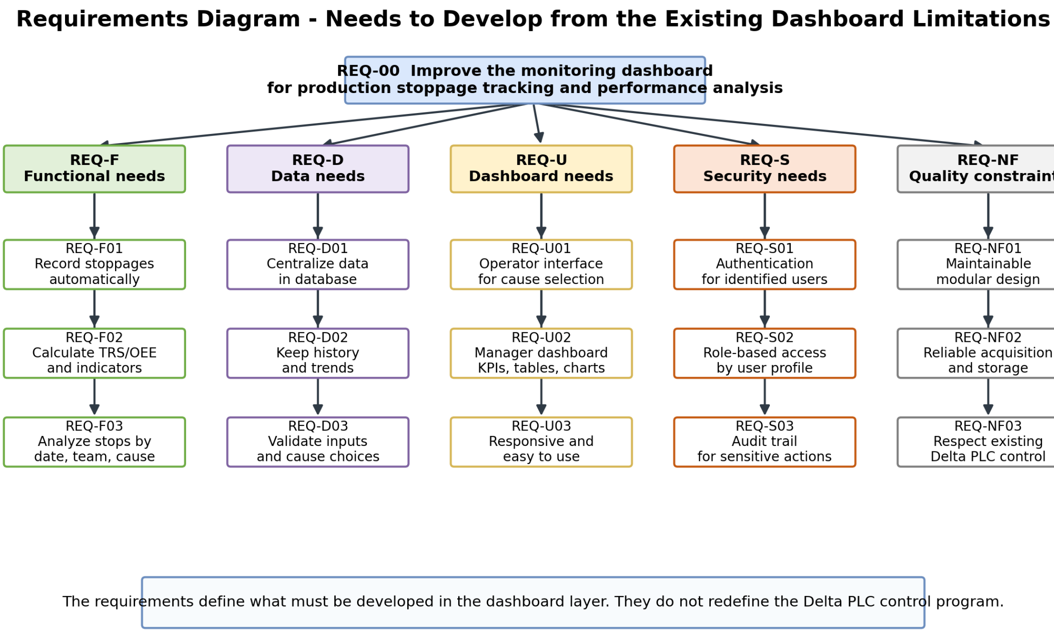

Figure 5: Requirements Diagram

2.3.1 Functional Requirements

Table 2: Functional requirements of the dashboard to be developed

ID	Requirement	Description
REQ-F01	Record stoppage events	The dashboard shall record the start time, end time and duration of each production stoppage.
REQ-F02	Qualify stoppage causes	The operator shall be able to select a stoppage cause from a controlled list of causes.
REQ-F03	Calculate indicators	The dashboard shall calculate TRS/OEE, total downtime, total productive time and number of stoppages.
REQ-F04	Display real-time information	The dashboard shall display the latest machine and production information without relying only on manual consultation.
REQ-F05	Analyze stoppage history	The manager shall be able to analyze stoppages by date, team, cause and duration.
REQ-F06	Compare periods and teams	The dashboard shall allow comparison between days, shifts or teams to identify recurring performance issues.
REQ-F07	Manage stoppage causes	Authorized users shall be able to add, modify, disable or delete stoppage causes.
REQ-F08	Generate reports and exports	The dashboard shall allow the export of production data and indicators for reporting and further analysis.

2.3.2 Non-functional Requirements

Table 3: Non-functional requirements and industrial constraints

ID	Requirement	Description
REQ-NF01	Maintainability	The future dashboard shall be organized so that business logic, interface logic and data access can be maintained clearly.
REQ-NF02	Scalability	The system shall support future extension of indicators, users and data volume.
REQ-NF03	Reliability	Data acquisition, storage and visualization shall be stable enough for production monitoring.
REQ-NF04	Performance	Dashboard pages and filters shall respond quickly enough for daily production follow-up.
REQ-C01	Preserve PLC control	The dashboard shall not disturb the validated control logic executed by the Delta PLC.
REQ-C02	Respect existing data sources	The system shall be designed according to the existing industrial data context: Delta PLC, Node-RED flows and MySQL data.
REQ-C03	Minimize operator burden	The dashboard shall not add unnecessary steps to the operator workflow.

3 Modeling languages: SysMl

SysMl, or Systems Modeling Language, is a graphical language used by MBSE (Model-Based Systems Engineering) practitioners to create system models and communicate ideas about system designs to stakeholders. It is a language that has a grammar and vocabulary consisting of graphical notations that have specific meanings. The purpose of SysML is to visualize and communicate a system's design among stakeholders [5].

The grammar and notations of SysML are defined in a standards specification published by the Object Management Group, which is a consortium of computer industry companies, government agencies and academic institutions. SysMl is an extension of a subset of the Unified Modeling Language (UML), and its complete definition requires referral to parts of the UML specification document as well [5].

We can name a few types of SysMl diagrams:

- **Behavior diagrams [5]**
  - Activity diagram
  - State machine diagram
  - Sequence diagram
  - Use case diagram

- Communication diagram

- **Structure diagrams [5]**
  - Block definition diagram
  - Internal block diagram
  - Parametric diagram
  - Package diagram
  - Composite structure diagram

- **Requirement diagram[5]**

Conclusion

Overall, chapter 1 has provided the necessary foundation for the subsequent chapters, enabling a comprehensive exploration of the problem and the development of an effective solution. By understanding the host company, the specific problem through the case study and the modeling language employed, you the reader will be equipped with the context and knowledge required to delve deeper into the next chapters presented in this report.

Chapter 2

Architecture and Design

Introduction

This chapter will give a glimpse on the conceptual phase of our project.

1 Dashboard

1.1 Use Case Diagram

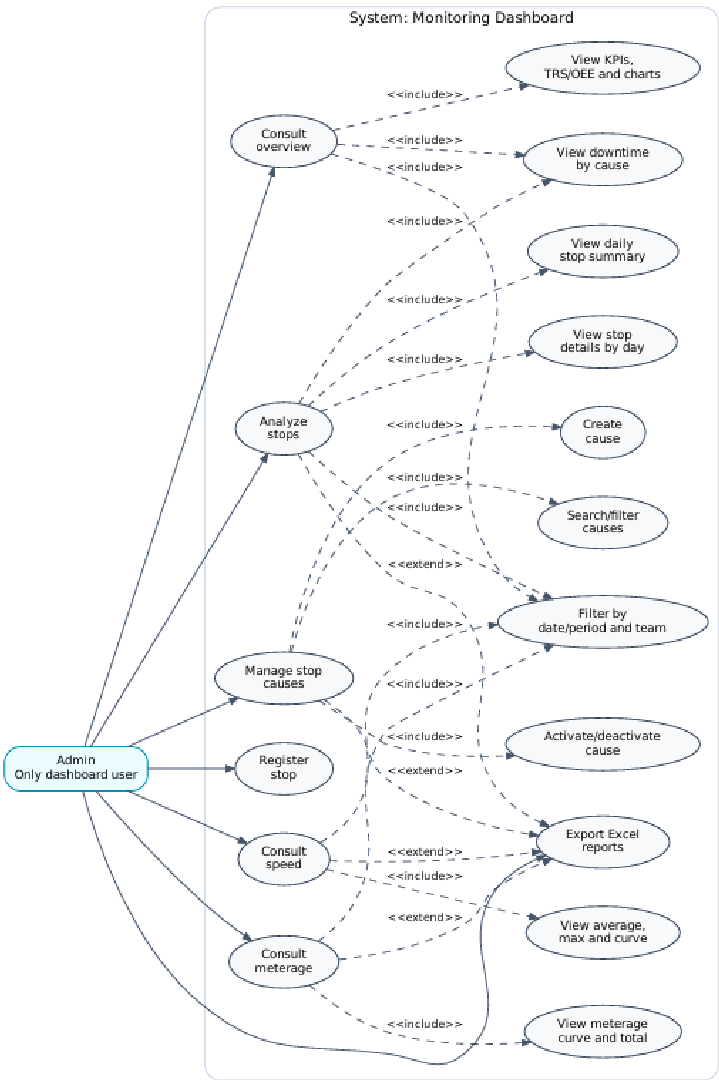

Figure 6: Use Case Diagram of the Dashboard

1.2 Use Case Description

Table 4: Use Case Descriptions

Use case	Actor	Description
Consult overview	Admin	Select a day or period, choose a team and view global KPIs and charts.
Analyze stops	Admin	Consult the daily summary, select a date, view paginated stop details and analyze downtime by cause.
Register stop	Admin	Create a stop through the back-end with a date, start time, optional end time and optional cause.
Manage stop causes	Admin	Search causes, create a cause, activate or deactivate a cause and export the cause list.
Consult meterage	Admin	View daily meterage and period total; export the data. Creation of meterage is not modeled.
Consult speed	Admin	View daily average/max speed and period summary; export the data. Creation of speed is not modeled.
Export Excel reports	Admin	Produce Excel files for stops, causes, meterage and speed.

1.3 Sequence - Overview Consultation

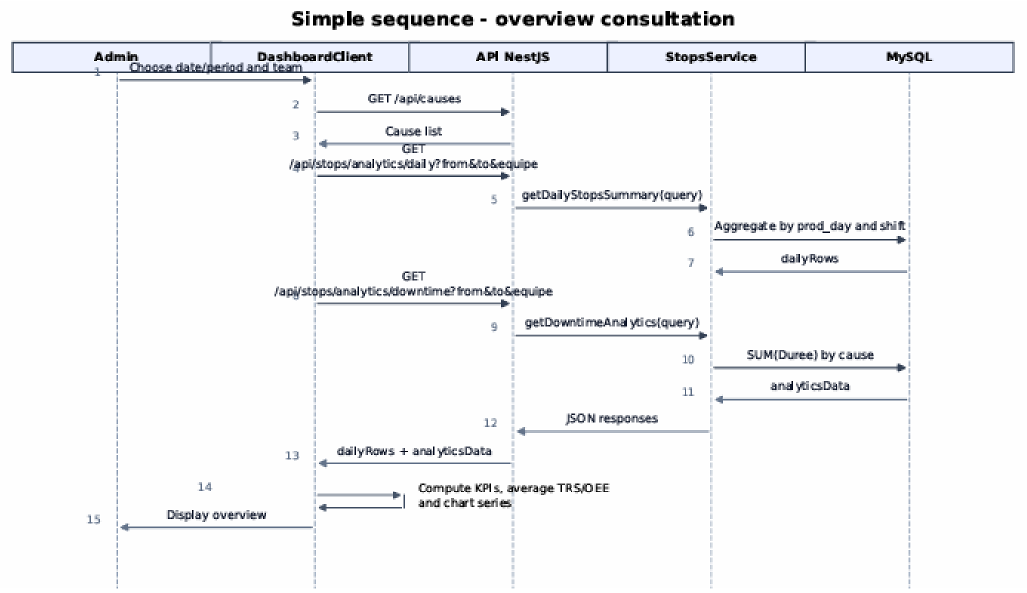

Figure 7: Sequence Diagram: Overview Consultation

1.4 Sequence - Stop Analysis and Excel Export

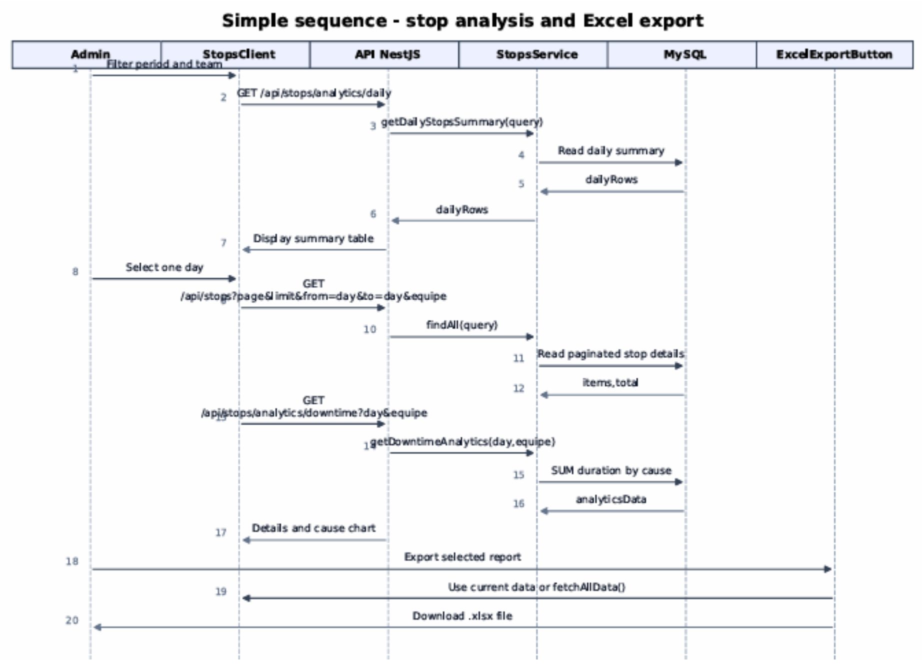

Figure 8: Sequence Diagram: Stop Analysis and Excel Export

2 Technical Revision Context

The machine is already installed in the workshop; therefore, the work is a CAD and documentation correction, not a new design. A mate error is treated here as a digital assembly inconsistency: failed reference, redundant constraint, over-defined relation, or wrong positioning.

The revision priorities are limited to the elements that affect technical reading, drawing extraction, and traceability. They are summarized in Table **??**.

Table 5: Scope of the technical revision

Revision item	**Technical action**	**Expected result**
CAD assembly mates	Correct failed, missing, redundant, or conflicting mates.	Stable rebuild with no detected mate errors.
Assembly organization	Keep the main machine groups in a coherent mechanical hierarchy.	Clear reading of unwinding, inspection, and rewinding functions.
Industrial title block	Standardize the sheet identification fields.	Traceable drawings with scale, revision, date, and validation data.
Technical drawing package	Prepare the useful views for the assembly, sub-assemblies, and parts.	Readable documentation for presentation, checking, and later updates.

3 Existing Machine and Functional Reference

The CAD model must keep the same fabric path as the real machine: input roll, unwinding zone, inspection zone, rewinding zone, and output roll. This functional reference gives a clear basis for grouping parts and drawing sheets.

The three main zones are kept in the documentation because they match the real mechanical operation of the machine. This avoids mixing all components in one unclear representation.

3.1 Complete assembly view

The complete assembly view is used as the global CAD reference of the corrected model. It shows the complete fabric inspection machine after the mate correction, with the unwinding block, the central inspection block, and the rewinding block kept in their real functional order.

4 CAD Assembly Correction

4.1 Mate review method

The mate review targeted constraints that could affect rebuild stability or technical reading. Only mates that define the mechanical function were kept; duplicated or contradictory mates were removed.

For rotating elements, the correction focused on coherent axes. For fixed supports and frames, it focused on stable contact references and logical fixation planes.

Table 6: CAD mate correction strategy

Observed CAD condition	**Correction approach**	**Technical purpose**
Failed or missing reference	Reassign the mate to a valid face, axis, plane, or reference geometry.	Restore stable positioning after rebuild.
Over-defined constraint	Remove duplicated or contradictory relations.	Avoid mate conflict and simplify the assembly tree.
Misaligned rotating element	Rebuild the concentric or coaxial relation on the correct mechanical axis.	Keep shafts, cylinders, bars, and bearings coherent.
Floating component	Add only the minimum functional constraints.	Prevent unintended movement without over-constraining the model.
Incorrect sub-assembly position	Reposition the group using stable references from the main structure.	Preserve the real machine architecture.

4.2 Final assembly validation

After correction, the assembly was rebuilt and checked. The validation was not limited to visual appearance; it also required a clean mate status, coherent axes, correct sub-assembly positions, and readiness for drawing extraction.

Table 7: Final CAD assembly validation criteria

Validation criterion	**Requirement**	**Expected result**
Mate status	No detected failed, dangling, or conflicting mates.	Clean assembly tree and reliable rebuild.
Component positioning	Main parts placed according to the real machine structure.	Correct visual interpretation of the layout.
Rotating axes	Cylinders, shafts, bearings, and bars aligned with coherent axes.	Consistent representation of rotating elements.
Sub-assembly hierarchy	Functional groups remain identifiable.	Clear reading of unwinding, inspection, and rewinding zones.
Drawing extraction readiness	Views generated from stable references.	Reliable basis for the technical drawing package.

5 Industrial Title Block Standardization

The title block was standardized so that each sheet can be identified without returning to the complete report. It gives the drawing its basic traceability: project, part or assembly name, drawing number, scale, revision, date, drafter, checker, and sheet number.

Table 8: Main fields of the revised industrial title block

Title-block field	Role	Reason for inclusion
Project or machine name	Identifies the global system.	Links the sheet to the fabric inspection machine.
Part or assembly name	Identifies the represented element.	Avoids confusion between similar components.
Drawing number	Gives a unique sheet reference.	Supports classification and retrieval.
Scale and sheet format	Defines representation conditions.	Makes the sheet easier to read and check.
Revision, date, drafter, checker	Records the current version and validation status.	Improves traceability and responsibility tracking.

6 Technical Drawing Package

6.1 Drawing package organization

The drawing package is organized by level: complete assembly, functional sub-assemblies, and individual parts. This avoids overloaded sheets and gives each drawing a precise technical purpose.

Table 9: Organization of the technical drawing package

Drawing level	Main content	Technical objective
Complete assembly drawing	Overall machine view, main zones, and principal components.	Present the corrected machine as a complete system.
Sub-assembly drawings	Separate views for unwinding, inspection, and rewinding groups.	Clarify the role and location of each functional zone.
Individual part drawings	Orthographic, section, detail, or isometric views as needed.	Identify each part and its useful geometry.
Title-block information	Drawing number, name, scale, revision, date, and validation fields.	Ensure traceability and professional presentation.

The CAD views for the chapter are named according to their mechanical function: complete assembly, unwinding unit, inspection unit, and rewinding unit. This naming keeps the drawing package readable because each view is linked to one clear documentation level instead of being treated as an isolated render.

6.2 Required views for the parts

The selected views must explain the geometry without unnecessary visual overload. Simple parts can be described with orthographic and isometric views; complex parts need sections or details when hidden geometry and interfaces are not clear.

Table 10: Recommended view selection for the drawing sheets

Component type	Recommended views	Purpose of the views
Complete assembly	General 3D view, front view, side view, and overall layout.	Show the complete machine and the main zones.
Functional sub-assembly	Isometric view, principal orthographic views, and local details if required.	Clarify composition and mechanical role.
Shafts, cylinders, and bars	Longitudinal view, end view, and interface detail if needed.	Show axes, diameters, ends, and mounting interfaces.
Supports, brackets, and plates	Front, top, side, and section views where necessary.	Make holes, thicknesses, fixing areas, and contact faces readable.
Moving or adjustable elements	Main view, position reference view, and connection detail.	Show mounting logic and adjustment interface.

7 Mechanical Reading After Correction

The corrected documentation should allow the reader to follow the fabric path and locate the main mechanical groups quickly. Each zone must therefore show only the elements needed to understand its function.

Table 11: Mechanical reading after CAD correction

Machine zone	Elements to represent clearly	Documentation objective
Unwinding zone	Input roll, expandable bar, bearings, drive elements, supports, and first guides.	Explain how the fabric is released and introduced into the machine.
Inspection zone	Guide cylinders, de-wrinkling device, inspection table, lighting support, and controls.	Show how the fabric is presented for visual inspection.
Rewinding zone	Output roll, expandable bar, drive elements, guide cylinders, actuators, and adjustment elements.	Explain how the inspected fabric is recovered into the final roll.

7.1 CAD views of the functional blocks

The following CAD views serve as sub-assembly references for understanding the functional blocks.

7.1.1 Unwinding Unit – Input Roll and Drive Support

The unwinding unit represents the inlet side of the machine. It supports the fabric roll, keeps the roll axis aligned, and links the roll to the drive and bearing supports. The isolated view makes the input roll, support frame, coupling area, and first guiding direction easier to identify.

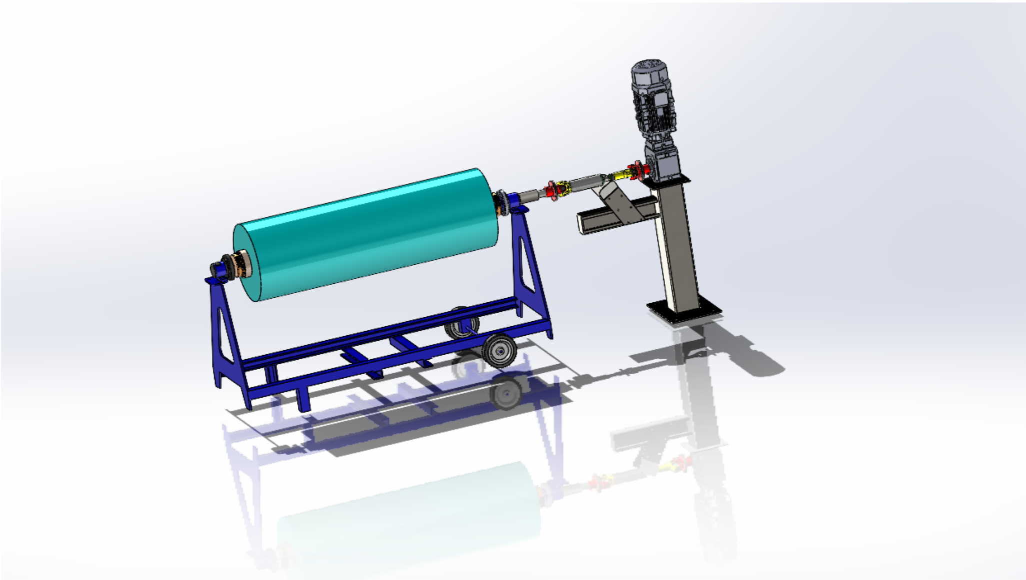

Figure 9: Unwinding Unit – Input Roll and Drive Support

7.1.2 Inspection Unit – Central Viewing and Guiding Section

The inspection unit is the central working area of the machine. It contains the main guiding rollers, the inspection opening, the support structure, and the elements that keep the fabric visible and correctly guided during checking.

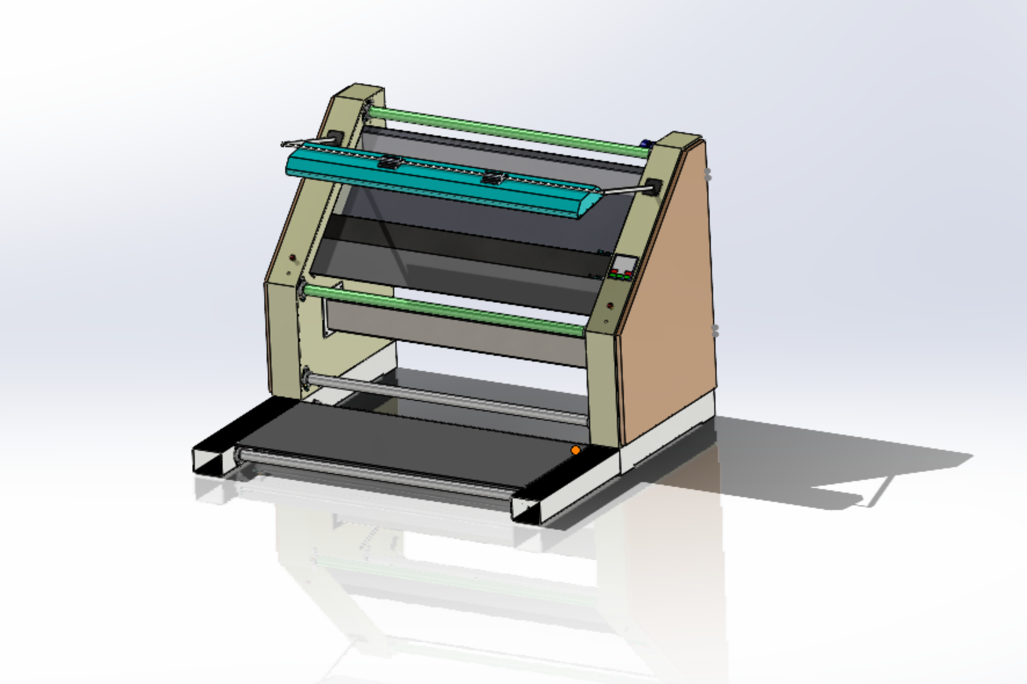

Figure 10: Inspection Unit – Central Viewing and Guiding Section

7.1.3 Rewinding Unit – Output Roll and Take-up Drive

The rewinding unit represents the output side of the machine. It receives the inspected fabric and winds it onto the final roll. The view highlights the take-up roll, drive support, bearing area, and connection with the previous guiding path.

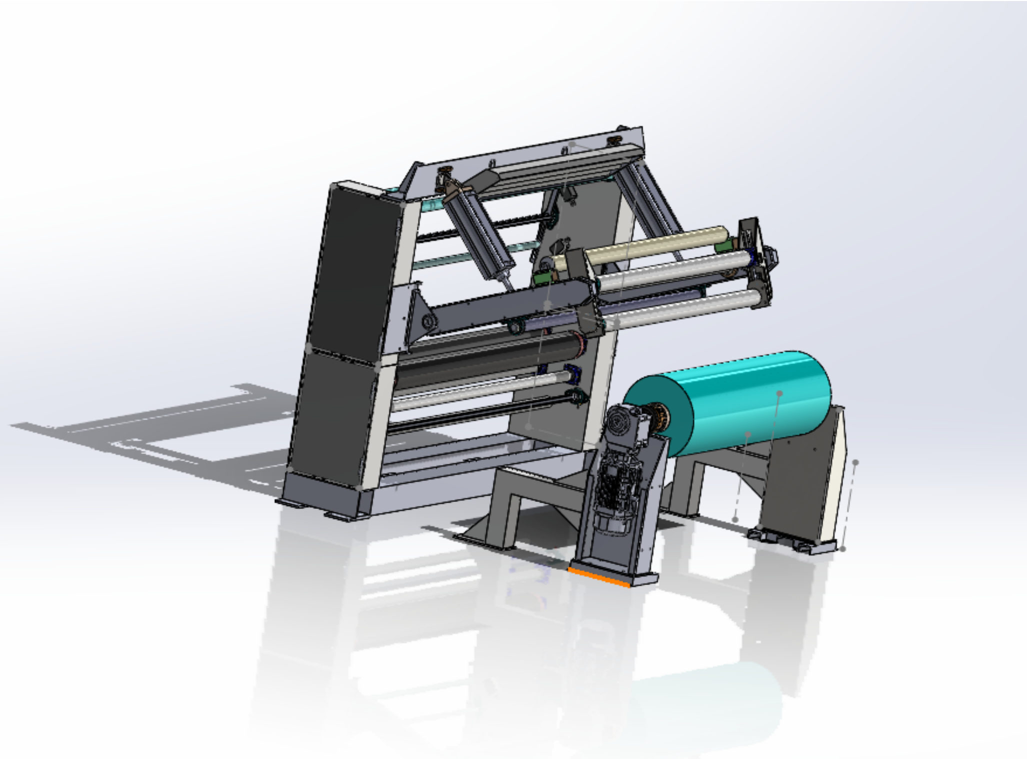

Figure 11: Rewinding Unit – Output Roll and Take-up Drive

8 Conclusion

The revision produced a cleaner CAD assembly, a coherent functional organization, and a standardized drawing package. The mate structure was checked for reliable rebuild and drawing extraction. The documentation improvements ensure traceability and readability.

Chapter 3

Realisation

Introduction

This chapter will give you a glimpse of what our dashboard has as well as our journey in optimising the database.

1 Dashboard Realisation

1.1 Overview Dashboard - Single-Day Mode

The overview dashboard is the main entry point of the application. In single-day mode, the Admin selects one production date and one team. The dashboard then displays the most important production indicators for that day, including the number of stops, total downtime, average TRS, and the number of analyzed days.

The page also provides graphical analysis through several charts. The downtime-by-cause chart allows the Admin to identify the causes that represent the highest production losses. Additional charts present the evolution of the number of stops, the comparison between downtime and working time, and the TRS evolution. This view gives the Admin a quick and synthetic understanding of the production situation for a specific day.

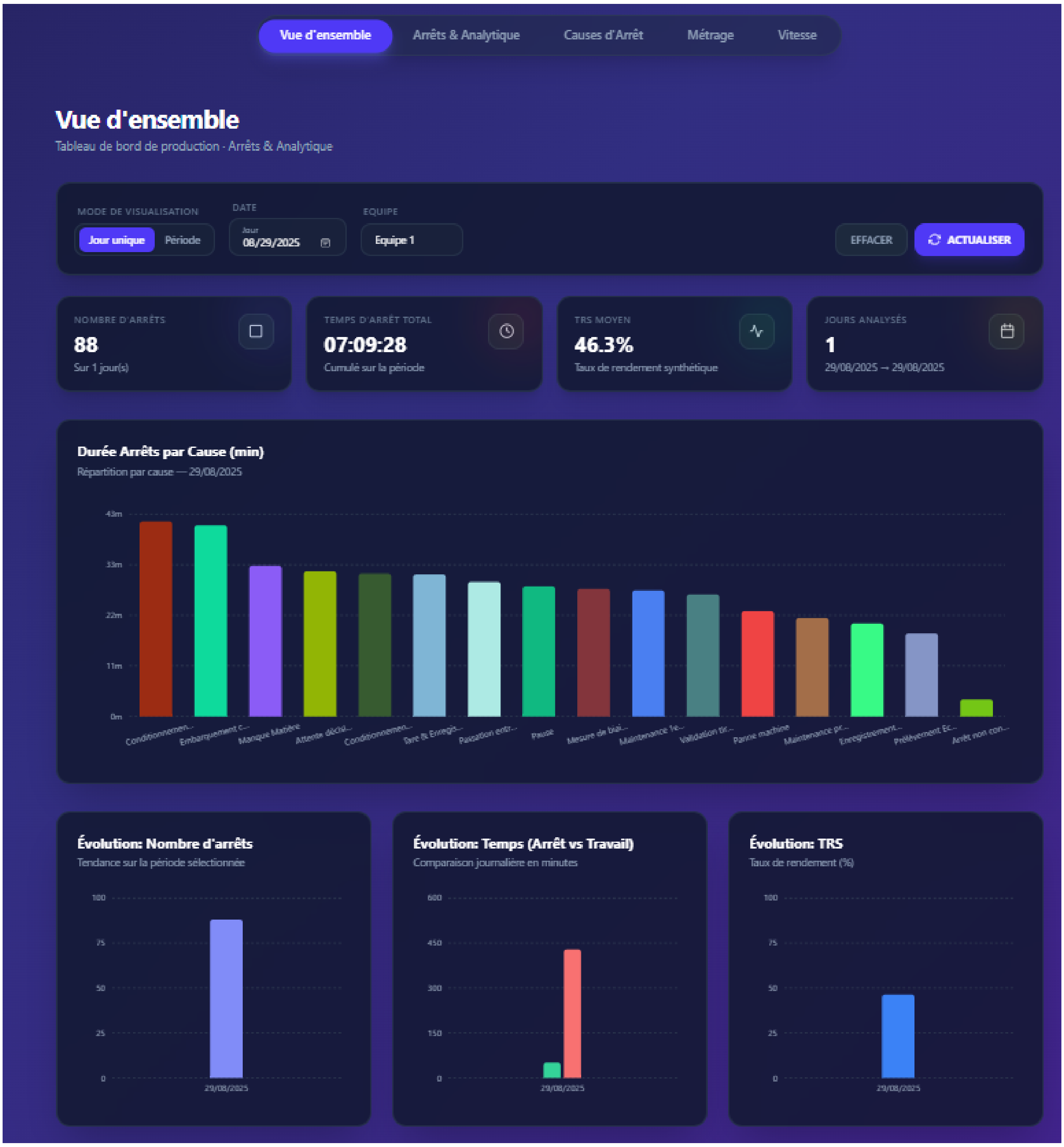

Figure 12: Overview Dashboard - Single-Day Mode Interface

1.2 Overview Dashboard - Period Mode

The period mode allows the Admin to analyze production performance over a selected date range. Instead of focusing on one day only, this view aggregates indicators across several days. The Admin can compare the evolution of the number of stops, total downtime, working time, and TRS during the selected period.

This page is useful for identifying long-term trends and recurring production issues. By visualizing stops and TRS over time, the Admin can evaluate whether production performance is improving, stable, or decreasing.

Figure 13: Overview Dashboard - Period Mode Interface

1.3 Stops and Analytics Page

The stops and analytics page provides a detailed analysis of machine stops. The Admin can select a visualization period and a production team. The page is divided into three main parts: a daily summary table, a detailed stops table, and a downtime chart.

The daily summary table displays the total stop time, total working time, TRS value, and number of stops for each production day. When the Admin selects a specific day, the detailed table shows each stop event with its start time, end time, duration, cause, TRS impact, and percentage contribution. This allows the Admin to move from a global daily view to a detailed operational analysis.

The bar chart groups total downtime by cause. This makes it easier to identify which causes generate the highest production losses. The page also includes pagination and Excel export features, which improve data consultation and reporting.

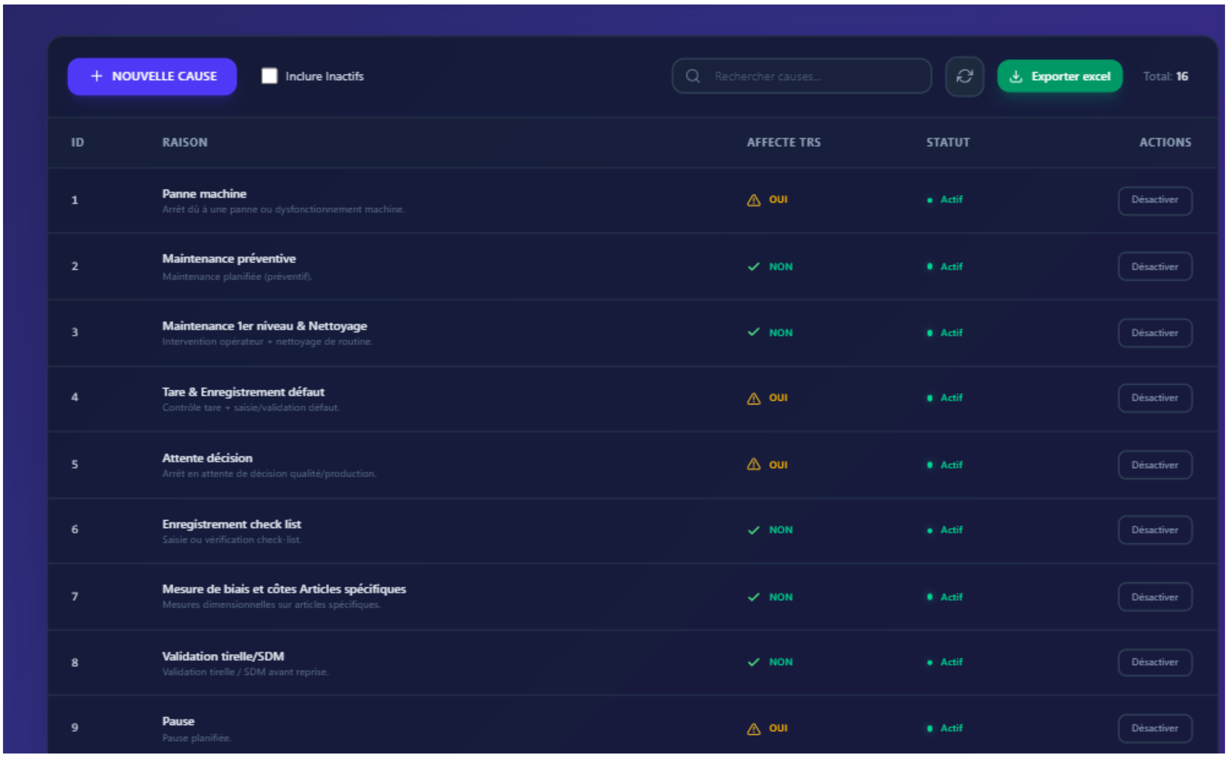

Figure 14: Stops and Analytics Module

2 Operational Context and Performance Metric

The production database contains four principal tables: `stops`, `causes`, `vitesse`, and `metrage`. The bottleneck analysed in this chapter comes from the `stops` and `causes` tables because they drive the downtime dashboard. Each row in `stops` records one machine-stop event with a stop date, start time, end time, and cause identifier.

Table 12: Core data model and workload constraints

Element	Description
Shift structure	Team 1 works 06:00–14:00, Team 2 works 14:00–22:00, and Team 3 works 22:00–06:00.
Production day	Stops before 06:00 belong to the previous production day, not to the current calendar day.
Workload	Approximately 800–1,500 stops are inserted per day by machine controllers; records are append-only.
Dashboard target	Supervisors query date-range views throughout the shift, so the practical target is interactive response time on production-scale data.
Main tables	`stops(id, Jour, Debut, Fin, cause_id)` and `causes(id, name, affect_trs)`.

The key computed field is the percentage of daily downtime represented by each stop within the same production day and team:

$$\mathrm{PCT} = \frac{\text{stop duration in seconds}}{\text{total downtime for the same production day and team}} \times 100. \tag{3.1}$$

This denominator is the central performance challenge. A naive query recomputes the total downtime for every returned row; the optimised design computes each total once and reuses it.

3 Baseline Diagnosis: Query A

The original query calculated duration, cause information, production-day assignment, and PCT directly in the `SELECT` clause. Its most expensive component was a correlated subquery that scanned `stops` again for each outer row in order to compute the PCT denominator. It also wrapped date columns in expressions such as `DATE_FORMAT(IF(...))`, which made the predicates non-sargable and prevented index use.

Table 13: Query A growth pattern

Rows	Runtime	Interpretation
100	0.291 s	Appears acceptable during early testing.
1,000	28.347 s	First clear warning sign.
2,000	114.156 s	Runtime grows by approximately four when the data doubles.
10,000	3,041.678 s	Already unusable for an operational dashboard.
14,000	5,512.934 s	Testing was stopped at this point.
1,000,000	≈326 days	Projection from the measured quadratic trend.

The measured behaviour follows:

$$T_A(n) \approx k \cdot n^2, \tag{3.2}$$

where n is the number of stop rows. Profiling confirmed the diagnosis: almost all execution time was spent inside the correlated denominator subquery, while table opening, logging, memory cleanup, and other phases were negligible. This made the optimisation target unambiguous: the PCT denominator had to be removed from the per-row execution path.

4 Optimisation Path

The optimisation work was deliberately staged. First, index-only fixes were tested to determine whether the existing query could be rescued. Next, the query was rewritten to eliminate the quadratic loop. Finally, the schema was redesigned so that the rewritten query could run with efficient ordered index scans.

4.1 Index-Only Experiments

Three variants of the same data were benchmarked with the original query shape. The results show the limit of indexing when the algorithm remains quadratic.

Table 14: *Index-only experiments on the original query shape*

Variant	Strategy	10K-row runtime	Conclusion
B1	Simple indexes on `Jour`, `Debut`, and `cause_id`	437.9 s	Ignored for the main filter because the production-day expression is non-sargable.
B2	Functional index on the production-day expression	451.7 s	The optimizer did not match the functional index reliably; overhead increased.
B3	Covering index on query columns	265.4 s	Faster scan source, but each outer row still triggers a full inner scan.

The index experiments establish a useful boundary: indexes can reduce constant factors, but they cannot change the complexity class. Because the correlated subquery still runs once per outer row, the query remains $O(n^2)$. At this stage the best improvement was approximately 37–40%, far below what production required.

4.2 Algorithmic Rewrite: Query C

Query C replaced the correlated denominator subquery with a derived table that pre-aggregates total downtime once per production day. The outer query then joins each stop row to the relevant precomputed total. This changes the execution pattern from repeated full scans to one aggregation pass plus one join pass.

Table 15: *Impact of replacing the correlated denominator with pre-aggregation*

Rows	B3 covering index	Query C	Result
1,000	4.679 s	0.094 s	49.8× faster.
5,000	100.702 s	0.287 s	350.9× faster.
10,000	265.4 s	0.373 s	711.5× faster.
1,000,000	≈31 days	46.642 s	Quadratic collapse removed.

Query C made the problem tractable, but it was still too slow for an operational dashboard at one million rows. `EXPLAIN ANALYZE` showed three remaining costs: repeated cause lookups, temporary-table materialisation for the aggregation, and repeated evaluation of production-day and duration expressions.

4.3 Window Function Test: Query D

A window-function version was tested to determine whether the denominator could be computed in a single pass using `SUM(Duree) OVER(PARTITION BY prod_day, equipe)`. It reduced the one-million-row runtime from 46.642 s to 25.955 s. However, the gain was limited because MySQL still had to sort and buffer rows by window partition. The lesson was that a query-level rewrite alone could not eliminate the remaining sort cost; the data had to be stored in an order that matched the dashboard aggregation pattern.

5 Final Architecture: Query E

The final solution moves repeated computations from read time to write time and makes the dashboard query read from a compact ordered index. It uses stored generated columns for production day, team, and duration, then applies a covering index aligned with the dashboard's filter and grouping keys.

5.1 Stored Generated Columns

Table 16: Generated columns introduced in the optimised schema

Column	Computation	Benefit
`prod_day`	Previous date when `Debut < '06:00:00'`, otherwise `Jour`.	Replaces repeated `DATE_FORMAT(IF(...))` and makes date filtering sargable.
`equipe`	Shift number derived from `Debut`.	Allows daily downtime to be grouped by the required production team.
`Duree`	Stop duration in seconds, including stops crossing midnight.	Avoids repeated `TIME_TO_SEC` and `CASE` expressions in every query.

This follows a write-once-read-many principle: the computational cost is paid once during insertion and reused by all dashboard reads.

5.2 Index Design

Table 17: Optimised index design

Index	Purpose
`PRIMARY KEY (prod_day, equipe, id)`	Clusters rows by the two dominant grouping keys while keeping each row unique through `id`.
`idx_id (id)`	Keeps `id` indexed for auto-increment and direct row lookup.
`idx_dashboard_covering (prod_day, equipe, cause_id, Duree, Debut, Fin)`	Covers the dashboard query, supports the date filter, and lets the aggregation stream through ordered key ranges.
`idx_cause_id (cause_id)`	Supports the join to `causes`.

The essential design principle is alignment: the first columns of the index match the date filter and the grouping keys. The aggregation therefore reads rows that are already ordered by `prod_day` and `equipe`, avoiding the disk-heavy sort observed in Query D.

5.3 Optimised Query

The final query keeps the successful pre-aggregation strategy but applies it to generated columns and an ordered covering index. The same date range is applied to both the aggregation and the final result so no totals are computed outside the dashboard scope.

```sql
WITH day_team_totals AS (
```

```sql
    SELECT
        prod_day,
        equipe,
        SUM(Duree) AS day_total
    FROM stops FORCE INDEX (idx_dashboard_covering)
    WHERE prod_day BETWEEN @from AND @to
    GROUP BY prod_day, equipe
)
SELECT
    s.Debut,
    s.Fin,
    s.Duree,
    s.prod_day,
    s.equipe,
    c.name AS cause,
    c.affect_trs,
    ROUND(s.Duree * 100.0 / NULLIF(dt.day_total, 0), 2) AS pct
FROM stops AS s FORCE INDEX (idx_dashboard_covering)
JOIN day_team_totals AS dt
  ON dt.prod_day = s.prod_day
 AND dt.equipe   = s.equipe
JOIN causes AS c
  ON c.id = s.cause_id
WHERE s.prod_day BETWEEN @from AND @to;
```

Listing 4.1: Final dashboard query using generated columns and pre-aggregated totals

6 Results and Discussion

Table 18: One-million-row benchmark summary

Query	Main technique	Runtime	Main limitation or outcome
A	Correlated denominator subquery	≈326 days projected	Full scan repeated per row.
B3	Covering index on original query	≈31 days projected	Scan is cheaper, but loop count unchanged.
C	Pre-aggregated derived table	46.642 s	Complexity fixed, but remaining expression and materialisation costs are high.
D	Window function	25.955 s	One scan, but partition sorting spills to disk.
E	Generated columns + covering index + CTE	2.200 s	Best measured result; reads ordered precomputed values.

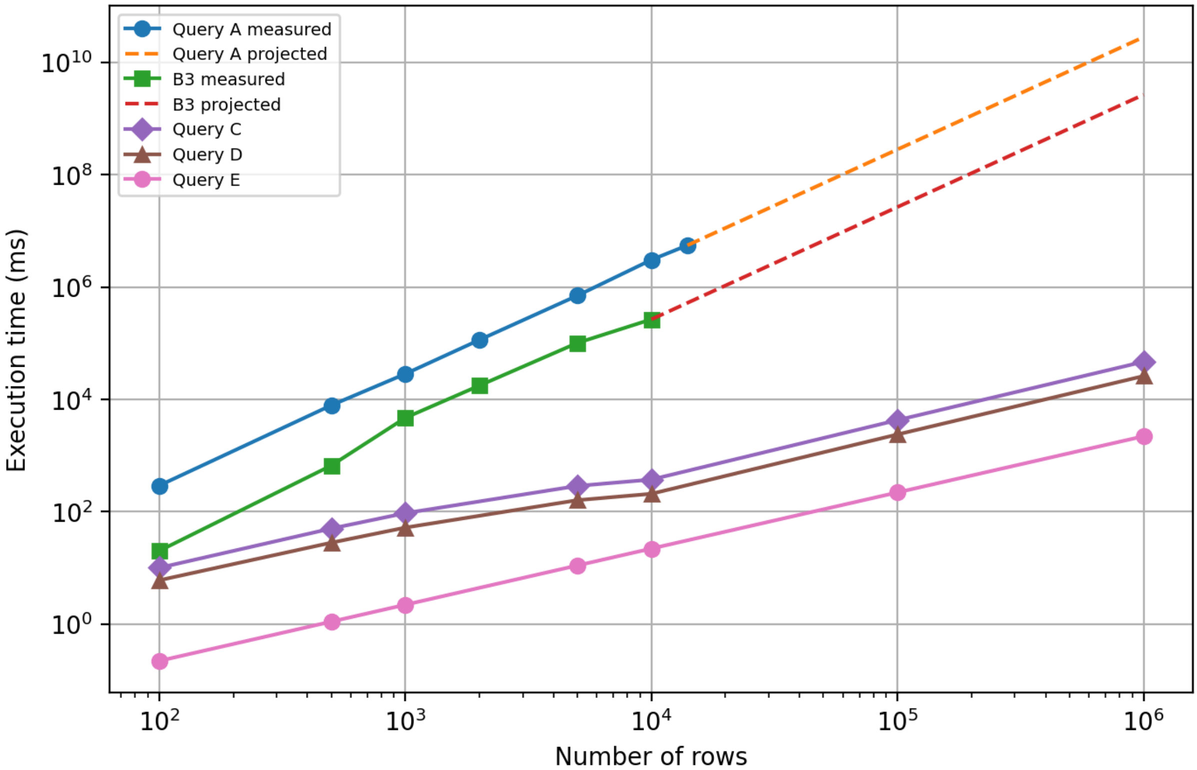

Figure 15: Figure 3.1: Execution-time trend across optimisation stages on a logarithmic scale

The final profile is fundamentally different from the baseline. Query A performs a nested scan and therefore grows quadratically. Query E performs a small aggregation over ordered generated values and then joins the compact totals back to the filtered rows. At one million rows, Query E is 21.2× faster than Query C, 11.8× faster than Query D, and several million times faster than the projected Query A runtime.

The main lesson is architectural. Indexes are valuable when the query already exposes searchable keys, but they cannot repair a per-row full-scan algorithm. The decisive gains came from changing where the work is done: production day, team, and duration are computed once at insert time, while daily team totals are computed once per dashboard query instead of once per returned row.

7 Conclusion

The optimisation journey reduced the dashboard query from a quadratic, non-sargable implementation to a linear, index-aligned design. The final schema and query preserve the business semantics of shifted production days, team-level downtime, stop duration, cause information, and PCT calculation, while making the database read precomputed values in the same order required by the aggregation.

This result confirms four practical principles for SQL performance engineering: avoid correlated aggregate subqueries on large fact tables; make filter expressions sargable; use generated columns for stable derived attributes; and design indexes from the actual filter, grouping, and projection pattern. The complete SQL listings and supporting schema definitions are provided in the technical appendix so the main chapter remains focused on the reasoning and evidence.

Chapter 4

Extension of the Supervision Architecture to Visiteuse 2

Introduction

This chapter presents the extension of the stop-supervision system to *Visiteuse 2*, a second production machine equipped with a Veichi VC3 PLC. The work focuses on adapting the PLC logic, Modbus TCP exchange, Node-RED processing, SQL insertion and HMI layers to this new machine.

1 Positioning of the Extension

1.1 Extension Beyond the Initial Specifications

The initial project focused on implementing a stop-supervision and stop-recording solution for one production machine. After this first implementation, the same functional principle was adapted to Visiteuse 2 in order to verify that it could be rebuilt on another PLC platform.

This extension represents an additional contribution to the project. It preserves the same operating principle: detection of a stop state, classification according to the stop duration, stop-cause selection, restart blocking for significant stops and database archiving.

The interest of this extension is both practical and technical. From a practical point of view, it gives the company a concrete example of how the supervision method can be reused. From a technical point of view, it required working with the Veichi VC3 PLC, the AutoStudio programming environment, the VI20 local HMI and a dedicated Node-RED communication profile.

1.2 Technical Objectives

The extension to Visiteuse 2 was structured around four objectives:

- develop a ladder program for the Veichi VC3 PLC while preserving the stop-recording principle used in the main project;
- configure Modbus TCP communication between Node-RED and the Veichi PLC according to the VC3 addressing convention;
- integrate the stop data into the existing SQL recording logic used by the project;

- provide two complementary interfaces: a VI20 local HMI near the machine and a browser-based web HMI for centralized supervision.

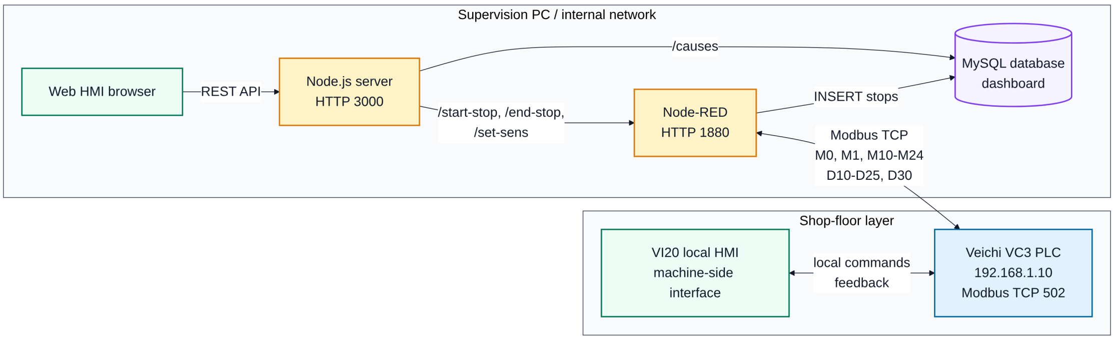

Figure 16: Deployment Architecture of the Supervision Extension Applied to Visiteuse 2

2 Implementation Methodology

2.1 Adopted Work Methodology

The implementation followed a layered sequence. The PLC layer was developed first because it contains the stop-cycle logic and the timestamp preparation. The communication layer was then configured in Node-RED to exchange data with the PLC. Finally, the local HMI and the web HMI were connected to the same functional logic.

The adopted development order was as follows:

1. programming the ladder logic of Visiteuse 2 in AutoStudio;
2. configuring Modbus TCP communication for the Veichi VC3 address map;
3. decoding the stop data and preparing the SQL insertion path;
4. developing the web HMI and its API calls through the Node.js server;
5. designing the local HMI on the Veichi screen using VI20;
6. validating the short-stop and long-stop scenarios through the HMI and SQL records.

Each layer has a defined role. The PLC handles real-time stop logic, Node-RED manages Modbus exchange and SQL insertion, the Node.js server exposes the web interface and REST API, MySQL stores the stop records, and the HMI screens provide operator interaction.

Table 19: Responsibilities of the main components used for Visiteuse 2

Component	Responsibility
Veichi VC3 PLC	Detects the stop cycle, applies the 30-second rule, manages restart blocking and prepares timestamp/cause data.
VI20 local HMI	Provides the shop-floor operator interface near the machine.
Node-RED	Reads and writes Modbus TCP variables, retrieves stop data and inserts stop records into MySQL.
Node.js server	Serves the web HMI, exposes REST endpoints and forwards web commands to Node-RED.
MySQL database	Stores stop records and cause labels used by the supervision system.
Web HMI	Provides centralized supervision and non-safety operator interaction from the internal network.

2.2 Preserved Functional Principle

The functional rule used for Visiteuse 2 is based on the same distinction between micro-stops and significant stops:

- a stop lasting less than 30 seconds is automatically classified with cause `16`;
- a stop lasting 30 seconds or more requires the operator to select a valid cause from causes `1-15` before the stop cycle is closed.

This rule avoids unnecessary manual recording of very short stops while keeping traceability for significant production losses.

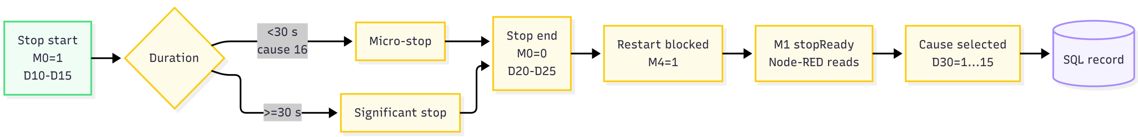

Figure 17: Functional Stop Scenario on Visiteuse 2

3 Programming and Control Logic

3.1 Ladder Development in AutoStudio

For Visiteuse 2, a new ladder program was developed in AutoStudio for the Veichi VC3 PLC. The program was not copied directly from the first machine; it was rebuilt according to the Veichi environment, its variables, its instructions and its timer behavior.

The ladder logic is organized around the following functions:

- using `isStopped` at address `M0` as the stop-state/control bit of the implemented supervision logic;
- recording the stop-start timestamp in `D10-D15`;

- resetting `D30`, `stopReady`, `causeSelected`, `restartBlocked` and cause bits at the beginning of a stop cycle;
- applying a 30-second timer to identify significant stops;
- blocking restart when a significant stop has no selected cause;
- converting cause bits `M10-M24` into the numeric cause identifier stored in `D30`;
- recording the stop-end timestamp in `D20-D25`;
- setting `stopReady` at `M1` so that Node-RED can read the prepared stop data.

In the current implementation, `M0` is named `isStopped` in the PLC variable table and is also written by the web command path through Node-RED. It is therefore described as an internal stop-state/control bit used by the supervision logic.

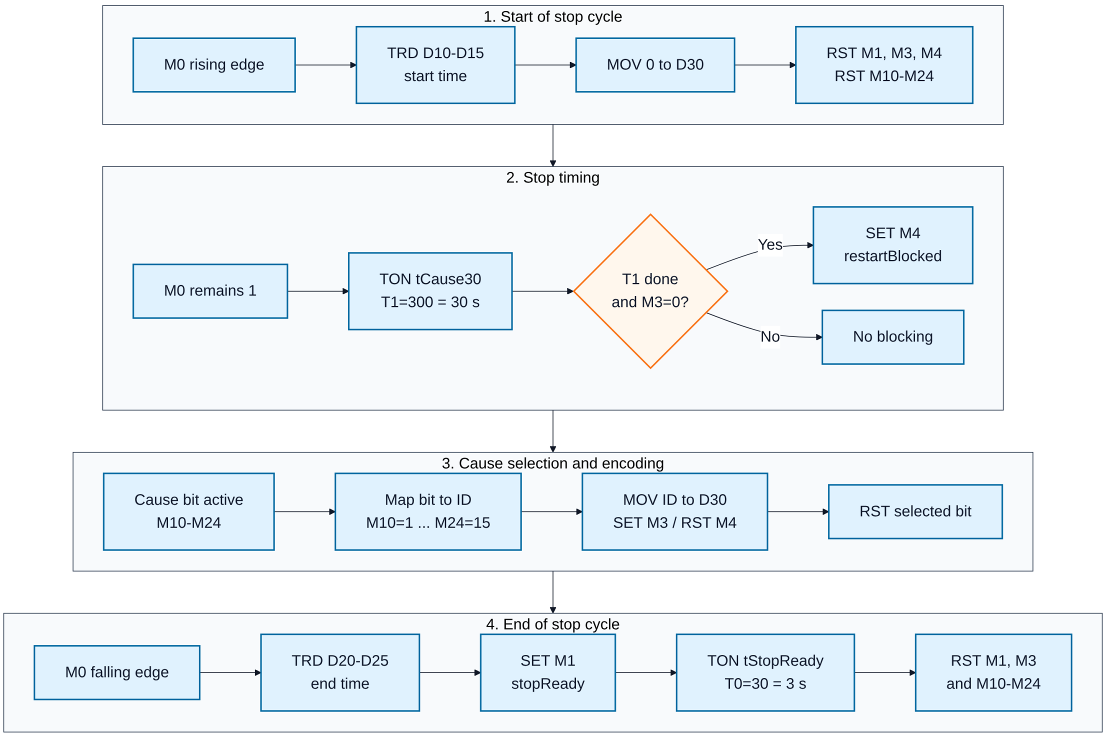

Figure 18: Logical Structure of the Ladder Program Developed in AutoStudio

3.2 Timer Adaptation Through TON Blocks

Only two timers are essential in this extension. The first timer, `tCause30`, detects whether the stop has reached the 30-second threshold. The second timer, `tStopReady`, keeps `M1/stopReady` active for a short period after the stop data have been prepared.

Table 20: Timer and synchronization parameters used in the implementation

Parameter	Value	Function
`tCause30`	`T1 = 300 = 30 s`	Threshold used to distinguish micro-stops from significant stops.
`tStopReady`	`T0 = 30 = 3 s`	Holds `M1/stopReady` long enough for Node-RED to detect the event.
Node-RED polling period	`500 ms`	Polling period used to read `M1`.
Stabilization delay	`100 ms`	Delay applied before extracting the timestamp and cause registers after `M1` becomes active.
Cause-to-restart delay	`200 ms`	Delay applied after writing the selected cause bit and before resetting `M0`.

3.3 PLC Synchronization and Variables Used

The synchronization between PLC control, HMI interaction and Node-RED communication is based on a limited set of bits, timers and registers.

Table 21: Main variables used in the Visiteuse 2 ladder program

Variable	PLC address	Functional role
`isStopped`	`M0`	Internal stop-state/control bit. It is set to `1` when the stop cycle starts and reset to `0` when the stop cycle is closed.
`stopReady`	`M1`	Indicates that the stop data are ready for Node-RED reading.
`tStopReady`	`T0`	Keeps `stopReady` active for approximately 3 seconds.
`causeSelected`	`M3`	Confirms that a cause has been selected during a significant stop.
`restartBlocked`	`M4`	Blocks restart authorization when the stop reaches 30 seconds and no valid cause has been selected.
`tCause30`	`T1`	30-second timer used to classify a stop as significant.
`M10-M24`	Cause bits	Bits used to transmit causes `1-15` to the PLC.
`D10-D15`	D registers	Stop-start timestamp generated by the `TRD` instruction.
`D20-D25`	D registers	Stop-end timestamp generated by the `TRD` instruction.
`D30`	D register	Numeric identifier of the selected cause for significant stops. For micro-stops, Node-RED archives cause `16`.

3.4 Timestamp Register Format

The `TRD` instruction stores timestamp fields in the order year, month, day, hour, minute and second. Node-RED reconstructs the start and end times from these registers before inserting the stop record.

Table 22: Timestamp register format used by Node-RED

Register	Field	Format used
`D10` / `D20`	Year	Numeric value. Two-digit values are normalized as `2000 + value`.
`D11` / `D21`	Month	Numeric value from `1` to `12`.
`D12` / `D22`	Day	Numeric value from `1` to `31`.
`D13` / `D23`	Hour	Numeric value from `0` to `23`.
`D14` / `D24`	Minute	Numeric value from `0` to `59`.
`D15` / `D25`	Second	Numeric value from `0` to `59`.

3.5 Stop-Cause Encoding

The stop causes selected by the operator are transmitted as bits. Each bit corresponds to one cause identifier. The ladder program converts the active bit into the numeric value stored in `D30`.

Table 23: Correspondence between cause bits and the value stored in `D30`

Bit	Cause ID	Bit	Cause ID	Bit	Cause ID
`M10`	1	`M15`	6	`M20`	11
`M11`	2	`M16`	7	`M21`	12
`M12`	3	`M17`	8	`M22`	13
`M13`	4	`M18`	9	`M23`	14
`M14`	5	`M19`	10	`M24`	15

Cause `16` is reserved for automatically classified micro-stops. It is not transmitted to the PLC as a manual cause bit.

4 Industrial Connectivity and SQL Integration

4.1 Node-RED Configuration for the Veichi Profile

After the ladder program was implemented, the Node-RED layer was adapted to communicate with the Veichi VC3 PLC. This step required a dedicated communication profile for the IP address, the Modbus TCP port, the unit identifier and the VC3 address mapping.

Table 24: Main communication parameters for the Veichi VC3 PLC

Parameter	**Value used**
PLC	Veichi VC3
PLC programming software	AutoStudio
PLC IP address	`192.168.1.10`
Protocol	Modbus TCP
Port	`502`
Unit ID	`1`
Modbus client name in Node-RED	`veichi-vc3-m-2000`
Modbus address of `M0`	Coil/protocol address `2000`
Modbus address of `M1`	Coil/protocol address `2001`
Modbus addresses of `M10-M24`	Coil/protocol addresses `2010-2024`
Timestamp and cause registers	Holding-register block starting at address `10`, with useful fields `D10-D15`, `D20-D25` and `D30`.

The main addressing difference concerns the `M` bits. In AutoStudio the variables are named `M0`, `M1`, `M10` and so on, whereas the Node-RED Modbus profile accesses them through Veichi protocol addresses: `M0 = 2000`, `M1 = 2001` and `M10-M24 = 2010-2024`.

4.2 Reading Stop Data

Node-RED periodically reads `M1/stopReady`. When this bit becomes active, the flow detects the rising edge, waits 100 ms and extracts the useful stop fields: `D10-D15` for the start timestamp, `D20-D25` for the end timestamp and `D30` for the selected cause.

Table 25: Useful registers extracted by Node-RED

Registers	**Data**	**Description**
`D10-D15`	Stop start	Year, month, day, hour, minute and second of the stop start.
`D20-D25`	Stop end	Year, month, day, hour, minute and second of the stop end.
`D30`	PLC cause	Operator cause identifier for significant stops.

If the calculated duration is less than 30 seconds, Node-RED archives cause `16`. If the duration is greater than or equal to 30 seconds, Node-RED uses the value read from `D30`, provided that it is between `1` and `15`.

id	jour	Debut	Fin	cause_id	Duree	prod_day	equipe
1	2026-05-22	00:59:24	00:59:25	16	1	2026-05-21	3
2	2026-05-22	01:04:46	01:05:10	16	24	2026-05-21	3
3	2026-05-22	01:05:22	01:06:06	9	44	2026-05-21	3

Figure 19: Example of Stop Records Inserted into the SQL Database

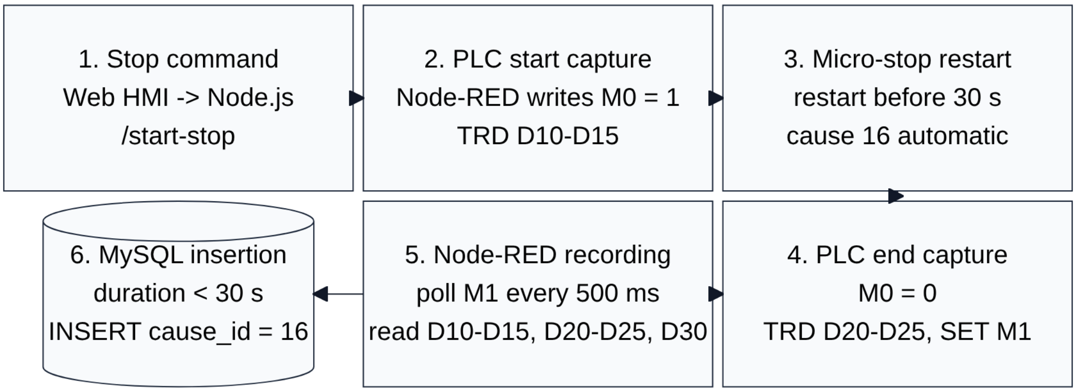

Figure 20: Short-Stop Communication Sequence Used for Automatic Micro-Stop Recording

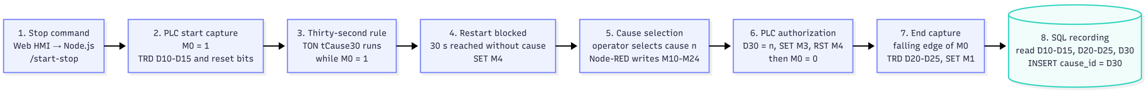

Figure 21: Long-Stop Communication Sequence with Mandatory Cause Selection

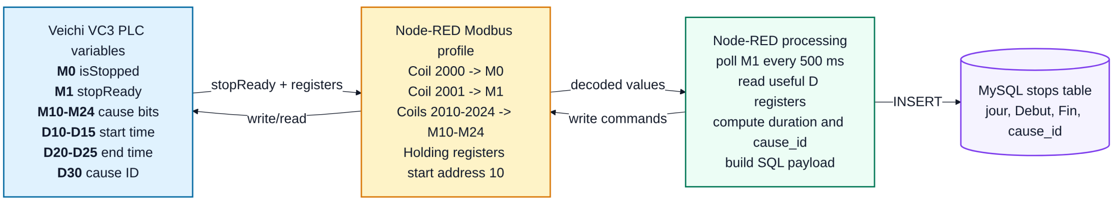

Figure 22: Mapping of Visiteuse 2 Variables in Modbus TCP Communication

4.3 Integration with the Existing SQL Recording Logic

The SQL database had already been implemented during the main project. For this extension, Node-RED reuses the existing stop-recording path and inserts the fields `jour`, `Debut`, `Fin` and `cause_id`. The remaining analysis fields, such as duration, production day and team, are computed by the existing SQL logic.

The web HMI also retrieves active cause labels from the database through the `/causes` endpoint. Cause `16` is excluded from manual selection because it is reserved for automatic micro-stop classification.

5 Development of Supervision Interfaces

5.1 Local Veichi HMI Developed with VI20

A local HMI was developed on the Veichi touch screen using VI20. This interface is intended for the operator positioned near the machine. It provides local supervision of Visiteuse 2 and direct access to the main operating commands.

Within this extension, the local HMI is complemented by the web HMI. The two interfaces are not alternatives: the local HMI remains the field interface, while the web HMI provides centralized access from the internal network.

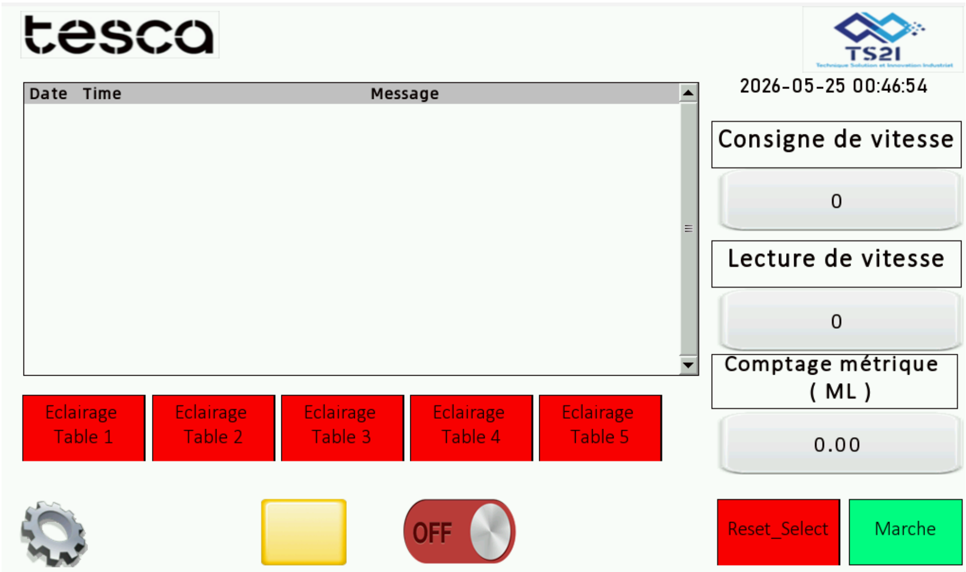

Figure 23: Veichi VI20 Local HMI - Main Operating Screen

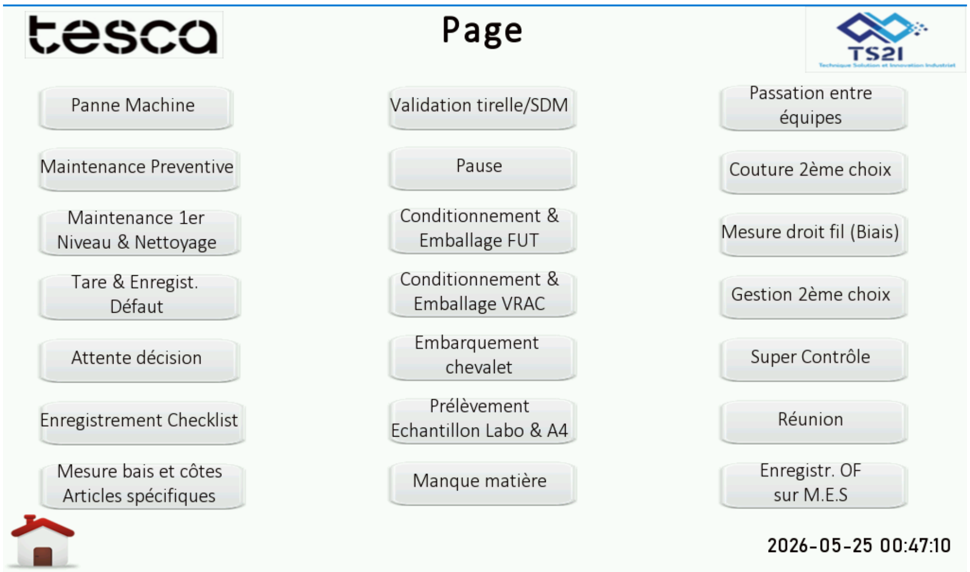

Figure 24: Veichi VI20 Local HMI - Stop-Cause Selection Screen

5.2 Centralized Web HMI

The web HMI was developed as a browser-based dashboard. It is served by the Node.js server and communicates with same-origin REST APIs. The Node.js server forwards validated commands to Node-RED, and Node-RED writes the corresponding Modbus coils or reads the required Modbus registers from the PLC.

The web HMI allows the user to send stop requests, follow restart authorization, display available causes and supervise the machine from the internal network. Safety-related functions remain handled by the PLC, the local safety devices and the existing machine safety chain.

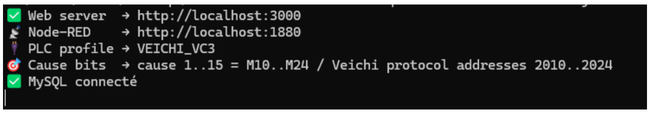

Figure 25: Runtime Status of the Web HMI Server and Communication Services

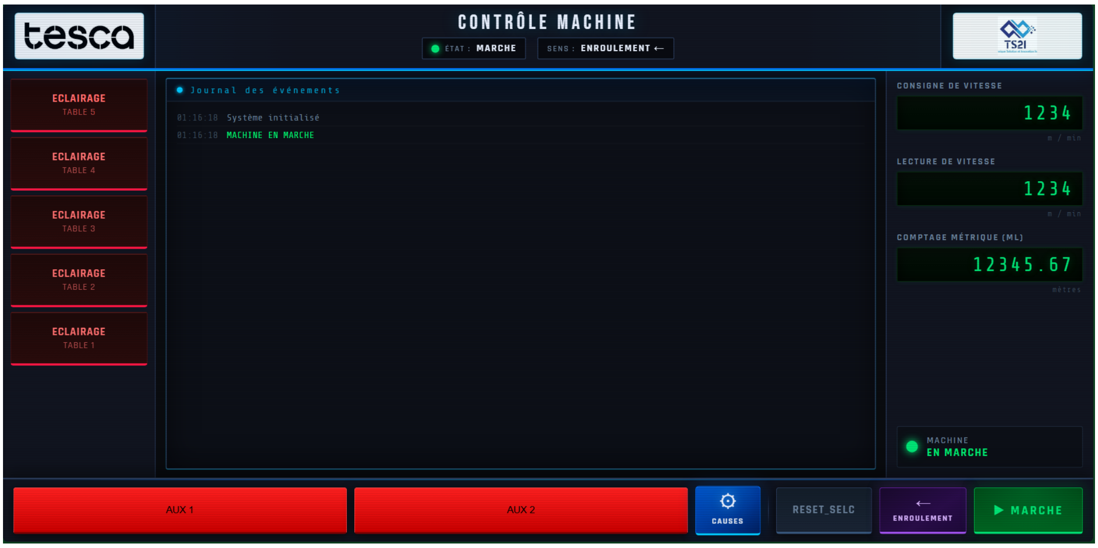

Figure 26: Web HMI - Supervision Dashboard

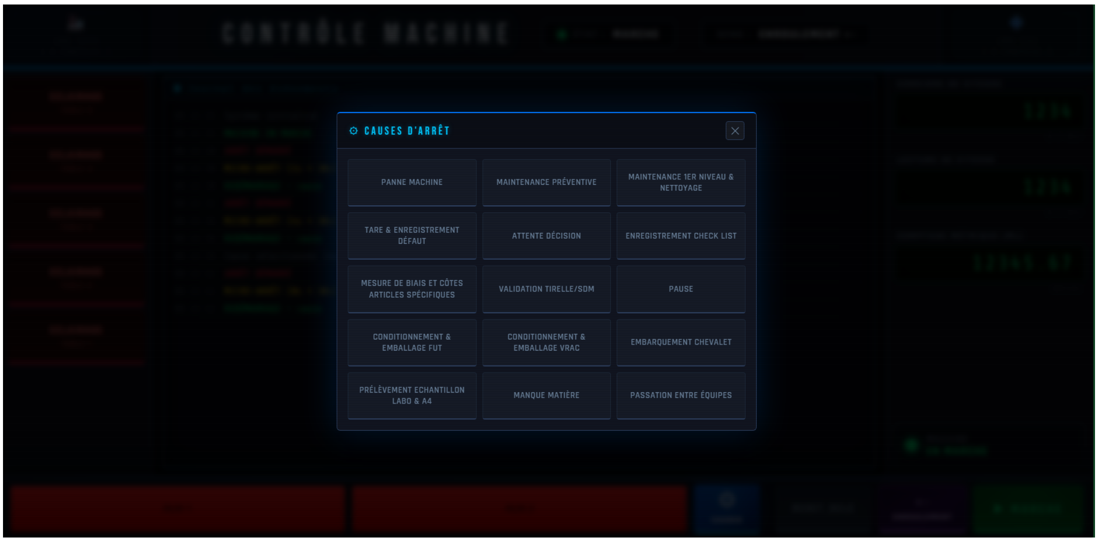

Figure 27: Web HMI - Stop-Cause Selection Window

Table 26: Main REST endpoints used by the web HMI

Endpoint	Method	Function
`/status`	`GET`	Returns the current web HMI state.
`/causes`	`GET`	Reads active causes from MySQL while excluding automatic cause `16`.
`/start-stop`	`POST`	Starts a stop cycle by writing `M0 = 1` through Node-RED.
`/end-stop`	`POST`	Sends the selected cause when required and closes the stop by resetting `M0`.
`/set-sens`	`POST`	Updates the selected winding direction; switching to `DEROULEMENT` triggers a stop cycle in the current implementation.

5.3 Complementarity Between the Local HMI and the Web HMI

The local HMI and the web HMI serve different operational needs. The VI20 screen is used close to the machine for shop-floor operation, while the web interface provides remote supervision from an internal workstation.

Table 27: Complementary roles of the two HMI layers

Criterion	VI20 local HMI	Web HMI
Location	Installed near the machine.	Opened from an internal-network workstation.
Main user	Shop-floor operator.	Operator or production monitoring user.
Main role	Local operation and cause selection.	Centralized supervision and non-safety commands.
Communication path	Direct interaction with PLC variables.	Browser, Node.js server, Node-RED and PLC.

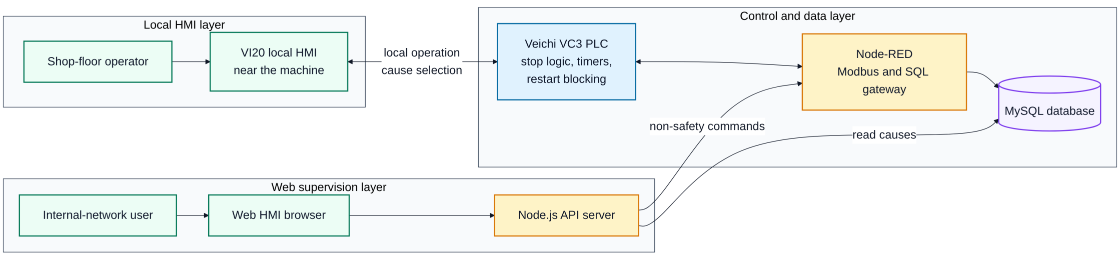

Figure 28: Complementary Roles of the Local HMI and the Web HMI

6 Functional Validation of the Extension

6.1 Test Scenarios

The extension was validated through the main operating scenarios of Visiteuse 2. The objective was to verify the consistency between the ladder logic, Modbus communication, HMI interaction and SQL recording. The main validation scenarios included short stops, long stops with cause selection, restart blocking, Modbus communication, web HMI operation and local HMI operation.

6.2 Results Obtained

The extension made it possible to deploy the stop-supervision principle on a second machine equipped with a different PLC platform. The ladder program was adapted to AutoStudio, the timers were implemented with TON blocks, the Modbus TCP profile was configured for the Veichi VC3 and the SQL insertion path was preserved.

The result confirms the feasibility of adapting the supervision method to Visiteuse 2. The implementation also provides a clear technical base for future reuse of the same principle, provided that each new machine receives its own PLC mapping and communication profile.

7 Contribution of the Extension to the Project

7.1 Technical Contribution

From a technical perspective, this extension made it possible to work on a new PLC platform and verify the portability of the stop-supervision principle. The transition to the Veichi VC3 PLC required timer adaptation, timestamp-register verification and precise configuration of Modbus addresses.

It also helped structure the system into clear layers: PLC logic, Node-RED communication, SQL recording, Node.js web API and HMI interaction.

7.2 Industrial Contribution

For the company, the main interest is the possibility of progressively reusing the same supervision principle on other machines. Visiteuse 2 serves as a concrete implementation example showing how the method can be adapted to another PLC while keeping a common operating logic.

The addition of the web HMI also improves usage flexibility. The operator keeps a local interface near the machine, while production users can access a centralized view from the internal network.

7.3 Personal and Pedagogical Contribution

On a personal level, this extension made it possible to go beyond a single-machine implementation. It provided experience with several technical environments and with the integration constraints between industrial automation, communication, database recording and web interfaces.

This experience strengthens the project because it demonstrates the ability to adapt an existing solution to a new industrial context.

Conclusion

The extension to Visiteuse 2 shows that the stop-supervision principle can be adapted to a second production machine equipped with a Veichi VC3 PLC. The final solution combines ladder logic, Modbus TCP communication, Node-RED processing, SQL recording and two complementary HMI layers.

Netography

[1] MRPEasy, "Overall Equipment Effectiveness (OEE): What It Is and How to Improve It," https://www.mrpeasy.com/blog/overall-equipment-effectiveness/, 2025, (Access date 04/02/2026).

[2] Automotive Industry Action Group (AIAG), "IATF 16949:2016 — Automotive Quality Management System Standard," https://www.aiag.org/expertise-areas/quality/iatf-16949-2016, 2016, (Access date 04/02/2026).

[3] FIPA-Tunisia — Foreign Investment Promotion Agency, "Automotive Sector in Tunisia," https://investintunisia.tn/en/automotive/, 2023, (Access date 05/02/2026).

[4] Tesca Group, "Tesca Group — Automotive Interior Manufacturer," https://www.tescagroup.com/en/, 2024, (Access date 10/02/2026).

[5] Object Management Group, "SysML: Systems Modeling Language," https://ptgmedia.pearsoncmg.com/images/9780321927866/samplepages/0321927869.pdf, 2013, (Access date 11/02/2026).